

Design-time business process compliance assessment based on multi-granularity semantic information

Xiaoxiao Sun¹, Siqing Yang¹, Chenying Zhao¹ and Dongjin Yu^{1*}

^{1*}School of Computer Science and Technology, Hangzhou Dianzi University, Hangzhou, China.

*Corresponding author(s). E-mail(s): yudj@hdu.edu.cn;

Abstract

Business process compliance is an essential part of business process management, which saves organizations from penalties caused by non-compliance processes. However, current researches on business process compliance mainly focus on checking using general constraint rules that have been formalized without in-depth analysis of related regulatory documents and mostly involve extensive human efforts. In this paper, we aim to propose an automatic and interpretable approach that encompasses the entire lifecycle of business process compliance. By combining deep learning and a natural language processing approach based on rule templates, we extract semantic information from regulatory documents at different granularities for subsequent compliance checking. In addition, we match appropriate rules to the process model and detect the degree of violation of the business process from three control-flow perspectives. The effectiveness of this method is validated on two real-world datasets.

Keywords: Compliance Checking, Violation Detection, Business Process, Text Mining, Natural Language Processing , Deep Learning

1 Introduction

Business Process Management (BPM) provides a framework to support the business process lifecycle from modeling, configuration and formulation to evaluation [1]. Business Process Compliance (BPC) is an essential part of business process management that measures how effectively an organization's business processes comply with all relevant laws, regulations, guidelines, and standards [2]. Examples of critical regulatory documents are the Health Insurance Portability and Accountability Act (HIPAA), the Sarbanes-Oxley Act (SOX), and the General Data Protection Regulation (GDPR) [3]. Companies that violate these regulatory documents risk losing the trust of investors, incurring financial fines, and facing criminal charges. As a result, adhering to rules from multiple sources has become essential for every organization to avoid huge fine losses and also to improve business process transparency [4]. However, in the company's practice, checking and ensuring the consistency of the organization's business processes with regulatory documents is still largely done manually with lot of efforts. In addition, the cost of manual review may increase significantly due to the constant changes in regulatory documents [5, 6].

BPC checking occurs at different stages of the process lifecycle [7]. During design time, process models are checked for compliance with all applicable constraints, thus preventing actual execution of non-compliance processes. At runtime, instances of the process being executed are continuously monitored to detect or even predict violations. Finally, in the audit phase, when the execution of the process instance is finished, BPC is checked against the log files generated by the system. This paper is dedicated to BPC checking at design time, i.e., checking violations of process models, as detecting any non-compliance at the early stage is crucial to saving the enterprise's effort, time, and financial resources [3, 5]. In previous studies, researchers have proposed several BPC methods targeted at process models. However, the checking constraints used by most approaches are a set of presupposed rules recorded using formal techniques such as Linear temporal logic (LTL) [8], Event calculus (EC) [9] or Compliance Request language (CRL) [10]. Although several researches aim to extract process descriptions or compliance constraints from regulator documents, formalizing legal specifications to precisely capture the semantics of the legal texts is still a huge challenge that requires significant human effort and complex modeling languages. Therefore, in this paper, we plan to propose a method that automatically extract semantic information from regulatory documents using natural language processing and machine learning methods, and further apply the information to perform BPC checking with process models at the design stage with interpretable results from multiple perspectives.

The contributions of our paper are the following:

1. We propose an automatic method that combines deep learning with rule-template-based natural language processing to extract semantic rules from regulatory documents at different granularities.

2. We disassemble the process model according to its structure to obtain detailed semantic information from its label text, and measure its compliance with extracted rules from (1) to detect three control flow types of violations.
3. The proposed BPC checking method covers the entire checking process and gives interpretable results with the warning of violation risks in the early-stage to avoid greater losses in the future for organizations.

The remainder of this paper is structured as follows. Section 2 discusses related works. Section 3 illustrates the problem and highlights the main challenges in this work. Section 4 describes our compliance assessment methodology. Section 5 evaluates and compares the proposed approach with existing methods on datasets and evaluates our approach on two real scenario datasets. Finally, Section 6 shows the conclusions and future work.

2 Related Works

In the last decade, various researches have been proposed to check the compliance or conformance of business processes. These techniques have been applied in many scenarios, including process queries, legal compliance checks, and audits.

In previous literature, several techniques for performing BPC checking at design-time have been proposed to check whether process models adhere to compliance rules, which are further classified according to their basic technologies. The first type is logic-based approaches, which applies formal logic modeling languages to formalize the specification, such as process compliance logic (PCL) [11], compliance request language (CRL) [10], event calculus (EC) [12], and other modeling languages. Pattern-based approaches translate compliance requirements into easy-to-understand visual patterns or graphics before actual BPC checking. For example, Elgammal et al. [10] used LTL-based graphical compliance patterns to model compliance constraints. In addition, hybrid approaches are also widely applied for BPC checking. Governatori et al. [13] applied SemanticWeb technology and languages such as LegalRuleML to manually model legal documents for compliance checking using Regorous, a business process regulatory compliance checker. Recently, Winter et al. [14] also evaluated compliance between regulatory documents and the BPMN models using simple textual analysis.

More recent attention has been focused on using advanced natural language processing (NLP) techniques to check for conformance between business processes and description text. For example, S'anchez-Ferreres et al. [15] used existing NLP techniques to tune textual descriptions and process models. Delicado et al. [16] introduced an online platform called NLP4BPM that converts textual descriptions into BPMN models and vice versa. The platform also computes the conformance between the textual descriptions and the BPMN model.

It is worth mentioning that the extraction of process constraints from natural language texts is usually performed as the essential part in BPC checking and is a central requirement for approaches that automatically extract process models from process-oriented texts [17]. Some approaches aim at eliciting process constraints from rule-oriented texts, such as requirements extraction from documents [18, 19] and declarative process constraints [20]. Specifically, some extract the structure of documents, while others classify legal passages based on their normative content [21] and distinguish between terms that belong to rules [22], and each claims varying degrees of success. Nevertheless, it is challenging to compare experimental results to the performance claimed in these researches because neither datasets nor systems are available to assess them. And according to a survey by Meth et al. [23], most of the existing methods are manual or semi-automatic.

In this research, motivated by a related issue investigated by Winter et al. [14], we intend to directly assess the compliance of process models using regulatory documents, i.e., pulls the required semantic information from regulatory documents with greater precision before multi-perspective matching with process models. Furthermore, the application of advanced NLP techniques addresses the problem of manually modeling compliance constraints in most BPC checking methods, considerably decreasing labor costs for companies.

3 Preliminaries

In this section, we present some preliminaries for better understanding of our method. Some basic definitions are given firstly, followed by problem definition of BPC checking.

3.1 Definition

Definition 1 (Process Model Repository, Process Model) A process model repository $M = \langle m_1, m_2, \dots, m_n \rangle$ contains multiple process models. A process model formally represents the business process within the organization in the form of graphical symbols. Specifically, a process model is a tuple $m = (A_m, E_m, G_m, R_m, N_m, F_m, L_m, u_m, t_m, f_m)$, where $N_m = A_m \cup E_m \cup G_m$ is a finite set of nodes, and A_m, E_m, G_m are a finite set of activities, events and gateways, respectively. R_m is a finite set of actors, and $F_m \subseteq N_m \times N_m$ is the flow relation and (N_m, F_m) is a connected graph. In addition, $u_m : A_m \cup E_m \rightarrow \{act(A_m \cup E_m), bs_obj(A_m \cup E_m)\}$ maps each activity and event with its action to a business object, while $t_m : G \rightarrow \{and, xor\}$ is a mapping that associates each gateway with a type, and $f_m : R_m \rightarrow A_m \cup E_m$ is a partial function that associates an actor $r \in R_m$ to an activity $a \in A_m$ or an event $e \in E_m$.

Definition 2 (Regulatory Documents Repository, Regulatory Document) A regulatory documents repository Γ contains regulatory documents from various sources, where each regulatory documents $e \in \Gamma$ is a tuple $e = (id, t, c)$, where id is the document id, t represents the title of the document, and c is the content of the document.

Definition 3 (Rule base, Rule record) Rule base R comprises of a set of rule records, where each $r \in R$ is denoted as $r = (t_r, A_r, P_r, C_r, O_r, E_r, U_r, f_r)$, where $t_r \in \{Obligation, Prohibition, Permission, Definition\}$ is the type of rule record. We utilize A_r, P_r, C_r, O_r, E_r to indicate a set of action, actor, condition, constraint and exception of a rule record r respectively. $U_r \in A_r \times A_r$ indicates the sequential relationship between the actions and a partial function $f_r : P_r \rightarrow A_r$ that specifies which actors must execute certain actions.

3.2 Problem Definition

The focus of this paper is to extract semantic information of different granularities from regulatory documents, which are further applied to finish BPC checking with process models from three different perspectives. The formal definition of our problem is as follows:

Input:

1. a process model repository: M
2. a regulatory documents repository: Γ

Middle Result:

1. semantic components of process model:
 $m \in M, m = (A_m, E_m, G_m, R_m, N_m, F_m, L_m, u_m, t_m, f_m)$
2. a rule base containing multiple rule record:
 $r \in R, r = (t_r, A_r, P_r, C_r, O_r, E_r, U_r, f_r)$

Output:

1. a score of missing action violation
2. a score of incorrect actor violation
3. a score of out-of-order execution violation

4 Methods

Figure 1 demonstrates an overview of our method to assess BPC between process models and regulatory documents at design-time, which consists of three parts.

1. **Process Model Disassembly:** This part extracts semantic information from the labels associated with the process model and extracts related control flow information. In particular, label parsing aims at extracting business components, i.e., actions and business objects, from activity and event labels.
2. **Regulatory Documents Parsing:** The purpose of the regulatory document parsing is to classify the constraint rules in the regulatory documents and extract semantic information from the constraint rules, thus creating a rule base containing a number of rule records. Here, a combination of deep learning and rule-based NLP method is applied from different granularities.

3. **Multiple Violation Detection:** Finally, we match the process model's semantic and control flow information with the rule records in the rule base to achieve BPC checking. In particular, our approach detects three types of business process violations, i.e., missing action violations, incorrect actor violations and out-of-order execution violations.

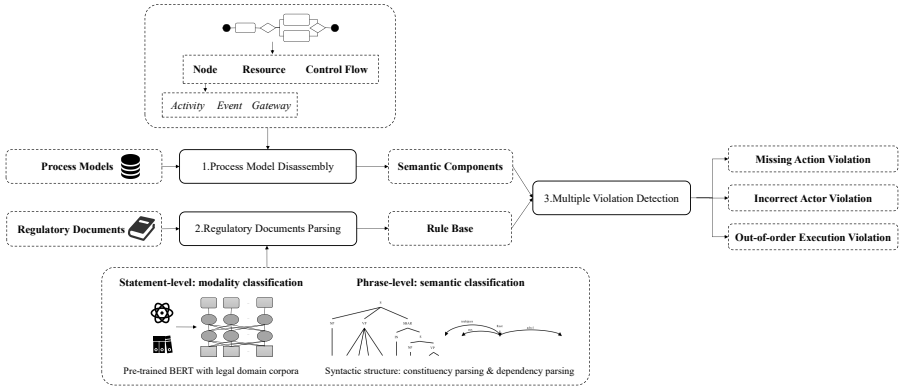


Fig. 1: Overview of the compliance checking approach

4.1 Process Model Disassembly

Design-time BPC checking aims to verify that business processes behave in a way that complies with all relevant rules, hence preventing the execution of non-compliance processes. Nowadays, organizations utilize a variety of modeling languages to document and standardize business processes, such as Petri nets, Unified Modeling Language (UML), and Event Driven Process Chain (EPC). In this paper, we mainly focus on parsing BPMN, one of the most commonly used notations for business processes. However, the proposed approach is generic and can be applied to other modeling languages.

As shown in Figure 2, a BPMN model consists of three types of nodes: events, activities, and gateways. Events indicate what happened, which are shown as circles. Activities are represented by rounded rectangles and are works performed in a business process. Gateways are responsible for controlling how business processes flow, shown as diamonds. These elements can be grouped into pools or channels based on the same actors (people, departments, organizations, etc.). We use the generic definition from Definition 1 to define the process model. The OR-gateway is not included in this description since it does not apply to all graph-based modeling notations (e.g., Petri nets) [24].

Here, we intend to extract information about activities, events, gateways, control flow relationships, and semantic label components from a given process model. Specifically, we take the following two semantic components out

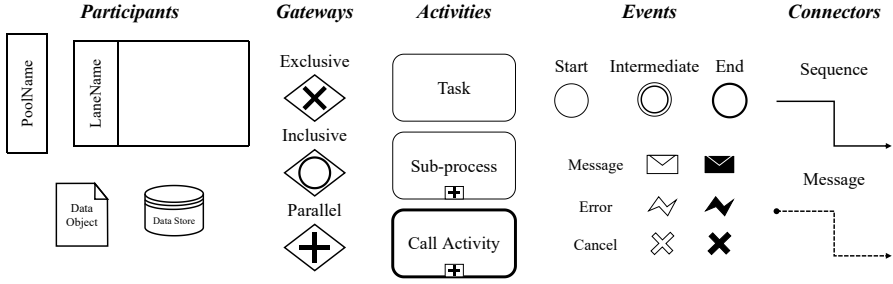


Fig. 2: Information extracted from process models and regulatory documents

of activity and event labels in order to extract their semantic content: the action describing the step, and the business object that the operation performs. The majority of the information we need can be obtained by applying Definition 1 because activities, events, gateways, and control flow information are all explicitly contained in the structure of the process model. However, further processing of the natural language information in the labels is required to extract the semantic components, namely actors, actions, and business objects. Standard NLP techniques provide inaccurate results since these labels are typically text snippets rather than complete sentences or phrases. In 2013, Leopold et al. [25] proposed a technique to decompose activity and event labels utilizing model context analysis and structural knowledge of labels. Here, we employ a similar technique to extract actions and business objects from the labels of process models, whose main steps are shown in Figure 3 and will not be extensively discussed in this paper due to space limitation.

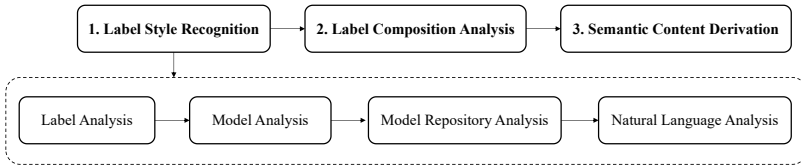


Fig. 3: Process label analysis

4.2 Regulatory Document Parsing

Regulatory documents do not explicitly describe activities, events, or gateways as process models. Therefore, all semantic information related to process model elements must be extracted from the text description using NLP techniques. In this part, the extraction is based on the smallest sentence unit of regulatory documents as our BPC checking approach attempts to align process model elements with each relevant rule record. In particular, we construct a rule base with specific components as Definition 3. To construct the rule

base, we extract semantic information of different granularities from regulatory documents, as shown in Figure 4. At the statement-level, the sentences from regulatory documents are classified into different categories of rule records, the method of checking the process models differs according to the different types of rule records in the subsequent BPC checking. At the phrase-level, phrases related to process model elements in rule records are extracted for matching and checking of process models.

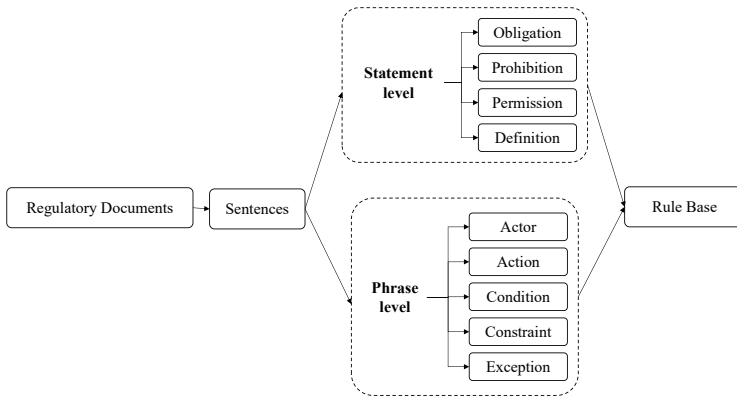


Fig. 4: Information extracted from regulatory documents

4.2.1 Statement-level: Modality Classification

Before extracting information from regulatory documents, the types of each sentence are defined and classified. The focus of this step is to identify definitional clauses and normative clauses. Definitional clauses are a general category with sentences that do not define the actor's behavior. Normative clauses are further divided into obligation, prohibition, and permission based on deontic logic concepts [26]. The detailed description of the four categories is shown in Table 1.

Table 1: The detailed description of the four categories of rule records

Concept	Definition
Obligation	It indicates a situation in an act or a course of action to which an individual is legally bound. Violation occurs when this condition is not met.
Prohibition	It indicates a situation in an act or a course of action that which an individual should avoid. Violation occurs when this condition is not met.
Permission	It expresses the permission to do something if the opposite obligation or its prohibition is not established. Violation occurs when this condition is not met.
Definition	It is a legal provision that defines the meaning of a concept.

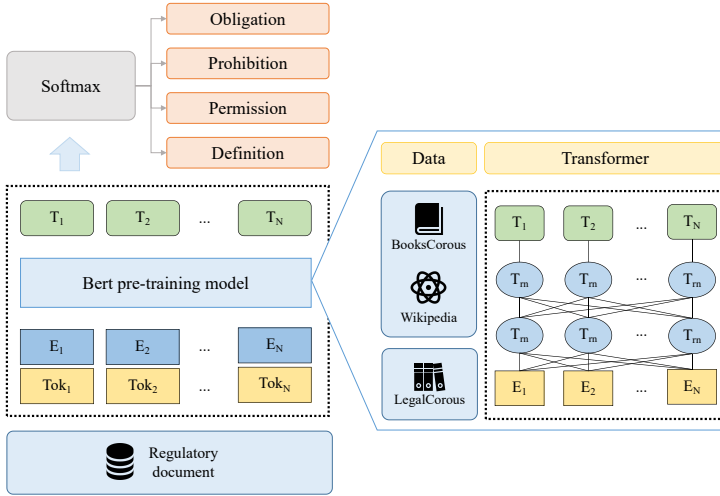


Fig. 5: Modality classification model based on BERT

In previous studies, the distinction between different categories was conducted based on modal verbs such as words like “must”, “should”, “may” and “should” as auxiliaries. However, in reality, rule records do not necessarily contain these modal verbs. For example, obligatory expressions would use “be instructed” and “be required” to indicate obligatory. To classify the rule records more accurately and efficiently, we employ deep learning to implement modal classification. As one of the most accurate language models for text classification tasks among the various deep learning models, the Bidirectional Encoder Representations from Transformers (BERT) model has received a lot of attention [27]. It is a large-scale pre-trained language model based on transformer that can be independently pre-trained on large datasets and then fine-tuned for the final target task in a specific domain and a specific task. Therefore, we utilize BERT and a corpus from the legal area to achieve sentence-level modality classification employing a pre-training fine-tuning strategy. The network is further fine-tuned and tested to see if it can accurately recreate the classification when applied to new data with labels. We outline the construction of the classification model in Figure 5.

In the pre-training stage, we use the general corpus and the legal corpus as the pre-training corpus data. In the fine-tuning stage, we perform supervised fine-tuning of the pre-trained model with a small amount of labeled data on our modality classification task. The fine-tuning process involves supervised training of the model in the labeled dataset, which changes the weight matrix of the model. The weight matrix of the model will change according to the training parameters, and multiple sets of training parameters are set to enable the model to achieve the best classification results under the datasets.

To apply pre-trained and fine-tuned BERT to our modality classification task, we add a classification layer (i.e., fully connected layer) after the output layer. The softmax function calculates the output of the classification layer to obtain the probabilities of the four classes of labels. The label corresponding to the maximum value of the final probability, i.e., obligation, prohibition, permission and definition, is the result of the modality classification.

4.2.2 Phrase-level: Semantic Classification

In this step, we extract constraint components that should be used to detect process model violations from the phrase-level. Since regulatory documents are written in formal languages using standardized language styles, we propose a method to accomplish this task using a query language on the syntactic parsing tree, based on self-defined rules and a lexicon of domain-marked words. Specifically, the method includes the following points:

1. Define phrase concepts that match process model elements in Section 3.1.
2. Analyze the text structure features of regulatory documents, discover the semantic rules for extracting defined phrase concepts, and perform multiple iterations on the semantic rules to improve the accuracy of semantic information extraction.
3. Develop a lexicon of marker words and extend legal domain-specific marker words to the basic lexicon.
4. Extract phrase-level semantic information from sentences by applying the combination of semantic rules and extended domain-marked lexicon, combined with classification results into rule records, and a rule base is constructed.

In order to achieve point (1), we first design six specific semantic concepts from the rules, i.e., modality, actor, action, condition, constraint, exception, whose detailed definition is shown in Table 2.

Table 2: Concept definitions

Concept	Definition
Modality	The auxiliary represents the action's modality.
Actor	The role is responsible for the execution of an activity.
Action	The content of the regulations, i.e. what is mandatory/prohibited/permitted.
Condition	The conditions of the regulation state when the regulation applies.
Constraint	The restriction is placed on the applicability of a legal provision.
Exception	The possible exceptions indicate when the provision does not apply.

According to the six semantic concepts defined, we analyze the text structure features of regulatory documents and formulate rules for extracting these semantic concept phrases from texts. Before using the rules for extraction, the text is parsed first. Syntactic parsing mainly includes constituency parsing and dependency parsing. While constituent parsing shows the whole sentence

structure and relationships, dependency parsing only indicates the relationships between words and their constituents. The following details the extraction rules that we design through analysis and iteration to ensure that the rules are as accurate as possible. The specific meanings of the symbols and abbreviations used in the rules are shown in Table 3. Also, Table 4 shows the marker words required in the rules.

Table 3: The meaning of the symbols and abbreviations used in rules

Notion	Meaning
A << B	A dominates B
A < B	A immediately dominates B
A \$ B	A is the sister of B
MD	Modal
NP	Noun phrase
VP	Verb phrase
IN	Preposition,
PP	Prepositional phrase
SBAR	Subordinate clause
SINV	Declarative sentence with subject-aux inversion

Table 4: Markers for different concepts

Concept	Examples of Markers
Actor	physician,expert,company,judge,prosecutor,driver,officer, inspector,...
Condition	if, in case of, provided that, in the context of, who,whose which...
Constraint	before, after, at least, at most, equal to, greatest, smallest, last of, least of...
Exception	with the exception of, except for, derogation, apart from,other than,...

Modality: Firstly, the modal auxiliary verb in the sentence is extracted, that is, the modality. The subsequent extraction of action phrases requires the modality, which is a crucial step. Rules identify modal (MD) nodes in the parsing tree that are dominated by verb phrase (VP). At the same time, VP is immediately dominated by the whole sentence or clause. The following are the modality extraction rules:

$$\begin{aligned}
 &(S < VP) << \mathbf{MD} \\
 &((SINV << S) < VP) << \mathbf{MD} \\
 &((SBAR << S) < VP) << \mathbf{MD}
 \end{aligned}$$

Actor: For actor extraction, the rules based on the constituent tree alone are not entirely accurate but require consideration of dependencies within the sentence. Actors must also appear in the subject or object dependencies defined by the rules. Also, the actor must be a noun phrase (NP). The specific rules are as follows:

$$\begin{aligned}
 &\text{subject dependency and } \mathbf{NP} < (\text{actor marker}) \\
 &\text{object dependency and passive voice and } PP < IN \$ (\mathbf{NP} < (\text{actor marker})) \\
 &\text{object dependency and active voice and } \mathbf{NP} < (\text{actor marker})
 \end{aligned}$$

Action: The single rule for action detects every verb phrase (VP) encountered, except those contained in modality, condition, constraint and exception.

Therefore, to work correctly, the rule for actions must run after the rules for the above four concepts:

VP with *modality, condition, constraint and exception phrases removed*

Condition: To detect the condition phrases, we use rules that check for nodes with a domination relation with nodes containing condition maker words such as “if” or “in case of”. Condition rules are:

SBAR <<(condition marker)

PP <<(condition marker)

Constraint: Constraints usually exist in the form of phrases in the sentence structure of English language. Therefore, it is a noun phrase that immediately dominates the constraint marker words, or a prepositional phrase that the constraint marker words to exist in the form of a preposition. The constraint extraction rules are as follows:

NP <(constraint marker)

PP <(IN <(constraint marker)) \$ NP

Exception: The extraction strategy for exception phrases is similar to that for conditions. The difference is that these rules consider exception marker words.

SBAR <<(exception marker)

PP <<(exception marker)

NP <<(IN <exception marker)

Specifically, we give an example to illustrate how to use the designed rules. A simplified constituent tree for this statement is shown in Figure 6. The following rule extracts the condition phrase in this statement: **SBAR** <<(condition marker). This rule matches any clause (SBAR) that contains a conditional marker (in our case, the term *if*).

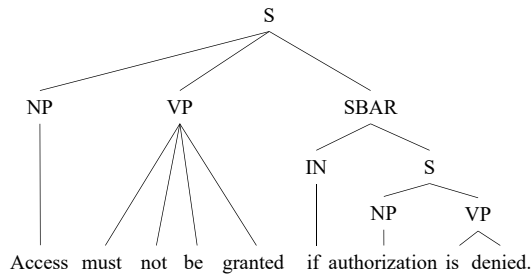


Fig. 6: Simplified constituent tree of the rule

For actor extraction, a simplified dependency tree for this statement is given in Figure 7. The rule extracts the actor phrases: *subject dependency and NP* <(actor marker). The rule classifies a noun phrase as an actor if it contains an actor marker and has a subject dependency (nsubjpass) with the main (root) verb in the sentence.

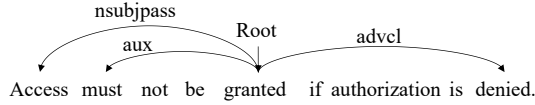


Fig. 7: Simplified dependency tree of the rule

It is worth noting that Table 4 illustrates the initial sets of makers for some of the concepts, and we extend the marker on these initial sets to make them as complete as possible. We observe that the scope of actors is broad, thus leading to a large set of potential markers. To refine the markers for actors, we analyze all entries in Wiktionary. For condition, constraint and exception, the markers come from our study, and the number of markers for these concepts is limited. Although it is likely that our study does not capture all possibilities, we expect that the markers for these concepts will rapidly saturate with use.

4.3 Multiple Violation Detection

After the semantic extraction of process models and regulatory documents, we conduct a variety of violation detection from the control flow perspective between the process model components extracted from Section 4.1 and rule records extracted from Section 4.2, which contains two main steps. First, we calculate matching score to indicate the association between the two and assign strongly associated rule records to individual process model. Secondly, the associated process model and rule records are checked for compliance from multiple perspectives to derive violation scores. In this section, we put forward several related definitions as follows.

Definition 4 (Matching Score) The matching score is used to measure the relevance between the process model and the rule records. Given two sets $D_{r,m}$ and $O_{r,m}$. $D_{r,m} \subseteq A_r \times action(A_m \cup E_m)$, $(a_r, a_m) \in D_{r,m}$ denotes that a_m is the action element in the event or activity of process model with the highest similarity to the element a_r of the rule record. $O_{r,m} \in P_r \times bs_obj(A_m \cup E_m) \cup R_m$, $(o_r, o_m) \in O_{r,m}$ denotes that o_m is the business object element in the event or activity of process model and actor elements with the highest similarity to the actor of rule records O_r . The matching score is defined as:

$$F(r, m, \tau) = \max \left(\frac{\sum_{(a_r, a_m) \in D_{r,m}} sim(a_r, a_m) > \tau}{|D_{r,m}|}, \frac{\sum_{(o_r, o_m) \in O_{r,m}} sim(o_r, o_m) > \tau}{|O_{r,m}|} \right)$$

where $sim(s_1, s_2)$ measures the similarity between two texts s_1 and s_2 , τ ranging from 0 to 1 is set to limit the degree of similarity.

Definition 5 (Missing Action Violation) $action_violation(r, m, \gamma)$ quantifies the non-compliance of actions in the rule record r that are not executed in the process model m .

$$action_violation(r, m, \gamma) = \frac{|\{(a_r, a_m) \in D_{r,m} \mid sim(a_r, a_m) < \gamma\}|}{|A_r|}$$

where γ ranging from 0 to 1 is set to limit the degree of action's similarity.

Definition 6 (Incorrect Actor Violation) $\text{executor_violation}(r, m, \vartheta)$ captures the actor violations in the process model m .

$$\text{actor_violation}(r, m, \gamma, \vartheta) = \frac{|\{r \in R_{r,m,\gamma} \mid \exists r' \in C_{r,m,\gamma}, \text{sim}(r, r') < \vartheta\}|}{|R_{r,m,\gamma}|}$$

$$R_{r,m,\gamma} = \{g \in P_r \mid \exists (a_r, a_m) \in D_{r,m}, f_r(g) = a_r \wedge \text{sim}(a_r, a_m) > \gamma\}$$

$$C_{r,m,\gamma} = \{r \in \text{bs_obj}(A_m \cup E_m) \cup R_m \mid \exists g \in R_{r,m,\gamma}, \text{sim}(f_r(g), f_m(r)) > \gamma\}$$

where $R_{r,m,\gamma}$ is the set of the actors in the rule records assigned to an activity for which there exists a equivalent task in the process model, C contains the actors in the process model that perform the same action as in the rule record, and ϑ ranging from 0 to 1 is set to limit the degree of actor's similarity.

Definition 7 (Out-of-order Execution Violation) The out-of-order execution violation scores between the model and the paragraph are calculated as follows.

$$\begin{aligned} \text{order_violation}(r, m, \gamma) \\ = \frac{|U_{r,m,\gamma}| - |\{(u_r, u'_r) \in U_{r,m,\gamma} \mid \exists \{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, (u_m, u'_m) \in F_m \wedge (u'_m, u_m) \notin F_m\}|}{|U_{r,m,\gamma}|} \end{aligned}$$

$$U_{r,m,\gamma} = \{(u_r, u'_r) \in U_r \mid \exists \{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, \text{sim}(u_r, u_m) > \gamma \wedge \text{sim}(u'_r, u'_m) > \gamma\}$$

where $(u_r, u'_r) \in U_r$ is a sequential constraint extracted from the rule record and $(u_r, u_m), (u'_r, u'_m) \in D_{r,m}$. $U_{r,m,\gamma}$ indicates that the similarity between the rule record and the operation in the process model is equivalent when the similarity is greater than the threshold γ . If the process model executes in the prescribed order, then there is only one path from t to t' in the model, i.e., $(u_m, u'_m) \in F_m$, while the opposite is not satisfied, i.e., $(u_m, u'_m) \notin F_m$.

The ability to measure the similarity between semantic rules extracted from the sentence and process models is critical to the performance of our approach. We use existing semantic similarity to calculate $\text{sim}(S_1, S_2) \in [0, 1]$ to refer to the quantification of the similarity between semantic components. Moreover, we set a threshold γ_{sim} such that if $\text{sim}(S_1, S_2) > \gamma_{\text{sim}}$, then the two semantic components are considered equivalent. Moreover, we have the flexibility to adjust the threshold γ_{sim} to adjust the degree of similarity required for two semantic components to be considered equivalent.

Based on the above similarity calculations between semantic components, we define a matching score in Definition 4, which aims to quantify how related a rule record is to the model. The higher the matching score, the more important it is to check the process model against that rule record. After getting the strongly correlated rules, we check the compliance of the process model based on these rules, which are calculated in Definition 5, 6, 7. According to the violation scores, the relevant personnel of the company can check the model in a targeted manner and make modifications to prevent subsequent non-compliance behaviors.

5 Experiment and Evaluation

This section presents our evaluation experiments, i.e., evaluation of modality classification, semantic classification, and compliance checking. Due to the fact that the process model disassembly is relatively simple, we will not conduct further evaluation of its result in this section. In the last part, we demonstrate a case study to show the results of our approach more clearly.

5.1 Evaluation of Modality Classification

5.1.1 Experimental Setup

This section describes the implementation, dataset, baselines, settings, and metrics used to perform the regulatory document modality classification evaluation experiment.

Implementation: We conduct our experiments on a 10-core Intel(R) Core (TM) i9-7900X CPU @ 3.30GHz and 64GB of RAM. In addition, the model is implemented in the PyTorch deep learning framework, configured with a CUDA environment of CUDA11.1 and the CUDNN7.5 deep learning library to accelerate GPU computation.

Dataset: The dataset used to train the machine learning model in this project is from the publicly available dataset [28]. The dataset is constructed from the publicly available Austrian Income Tax Code¹. The sentences in the collected text were divided into four categories and labeled 0-3 for each of these four categories, i.e., definition, obligation, prohibition, and permission. We randomly divided the dataset into a training set and a test set in a ratio of 9:1 to have as many sentences as possible in the test set.

Evaluation metrics: We use Precision, Recall and F-Score to measure the classification results, which are accurate measures of model performance even when the data is skewed. These metrics are defined as follows:

$$\begin{aligned} Precision &= \frac{TP}{TP + FP} \\ Recall &= \frac{TP}{TP + FN} \\ F1 - Score &= \frac{2 * Precision * Recall}{Precision + Recall} \end{aligned}$$

where TP is the number of true positives, FP is the number of false positives and FN is the number of false negatives.

5.1.2 Classification Results

In order to demonstrate the superiority of our pre-training method with legal corpus and fully examine the effects of different pre-trained models on classification results, we conduct a comparative analysis with all the pre-training models, i.e., domain pre-trained models “bert-legal-cased” and

¹Available at <https://ris.bka.gv.at/eli/bgbl/1988/400/P0/NOR40205159>

Table 5: Structure and parameters of different pre-trained models

Pretraining Data	Model	Layers	Hiddens	Parameters
BookCorpus and English Wikipedia	bert-base-uncased	12	768	110M
	bert-base-cased	12	768	110M
	bert-large-uncased	24	1024	340M
	bert-large-cased	24	1024	340M
Documents of EU legislation	bert-legal-uncased	12	768	110M
	bert-legal-cased	12	768	110M

Table 6: Classification results of different pre-trained models

Model	Precision	Recall	F1
bert-base-uncased	90.8%	89.6%	88.7%
bert-base-cased	86.7%	87.6%	84.1%
bert-large-uncased	89.4%	91.3%	86.7%
bert-large-cased	88.8%	89.7%	89.0%
bert-legal-uncased	92.1%	94.1%	89.3%
bert-legal-cased	91.3%	93.3%	88.5%

“bert-legal-uncased” and other pre-trained models “bert-base-cased”, “bert-base-uncased”, “bert-large-cased” and “bert-large-uncased”. The pre-trained models ending with “uncased” converts all input text to lowercase to reduce the effect of letter case on the classification effect compared with pre-trained models ending with “cased”. The “bert-large-uncased” model contains more parameters than the other models. Table 5 compares the pre-training data, structure, and parameters of these pre-trained models.

Moreover, to illustrate the superiority of the performance of the classification model obtained after fine-tuning the pre-trained BERT model, we compare our method with the following three commonly used classification methods:

CF_{KW} : This baseline uses a traditional classification method that simply uses keywords to determine the type of text.

CF_{RNN} [29]: This baseline utilizes RNN for text classification. It is common to use RNN and its extended models to process text sequence models. Here we use bidirectional LSTM for text classification.

CF_{CNN} [30]: This baseline utilizes CNN for text classification. CNN is a class of deep, feed-forward artificial neural networks that use a variation of multilayer perceptrons designed to require minimal preprocessing.

Table 6 compares the results of the different pre-trained models after fine-tuning. The results in the table show that the pre-trained model using with legal corpus and ignoring letter case works best, while increasing the complexity of the model does not significantly improve the classification results.

We compare the classification method using the pre-trained model “bert-legal-uncased” with other methods, and the results are presented in Table 7. From the results in the table, it can be concluded that our method outperforms other methods in terms of precision, recall and f1-score, where the

Table 7: Classification results of different model methods

Model	Precision	Recall	F1-score
CF_{KW}	62.7%	64.7%	60.8%
CF_{RNN}	90.3%	91.6%	90.9%
CF_{CNN}	87.4%	90.1%	90.2%
Our classification approach	92.1%	94.1%	93.1%

lowest accuracy is CF_{KW} achieved using the keyword method. BERT is a bi-directional language model with attention mechanism, which can better focus on the contextual relationship in the text. The model has more vital language expression ability through pre-training with extensive corpus data than traditional machine learning algorithms. At the same time, our method uses domain-specific corpora for pre-training to achieve better results than other deep learning methods.

5.2 Evaluation of Semantic Classification

This section describes the experimental setup of the method used to extract semantic components from sentences of regulatory documents and the presentation of the results.

5.2.1 Experimental Setup

Implement: Our extraction rules were implemented using Tregex patterns, Tsurgeon operations [31], and Java. These rules are parsed using the output of a classical NLP pipeline. They are fully automated and require no human intervention to preprocess sentences. The Stanford CoreNLP [32] pipeline is used to perform typical NLP pre-processing tasks, including tokenization, sentence segmentation, part-of-speech tagging, and constituency parsing.

Dataset: Similarly, we constructed the dataset using the Austrian income tax law used in the previous experiments. We selected 150 of these sentences for annotation as ground truth. The selected sentences were typically complete, sequential, comprised a legal act, and were longer than 20 words. One of the authors annotated 150 sentences used in the evaluation, and another author independently annotated 10% of these statements to check the reliability of the manual annotations.

Evaluation metrics: Each phrase-level annotation has two parameters: type and span. The former specifies the concept and the latter specifies where the phrase starts and ends in the statement. We evaluate the results of automatic phrase-level semantic information extraction using the following concepts: A phrase is matched if the computed span of the phrase has a non-empty intersection with some ground-truth phrase of the same type. If the calculated annotations do not match, they are *misclassified*. Ground-truth phrases that do not match are considered as *missed*.

5.2.2 Classification Results

For each concept, we provide the number of matches, the number of misclassified phrases, the number of missed phrases, and the precision and recall scores. Each matching phrase counts as a true positive phrase (TP); each misclassified phrase counts as a false positive phrase (FP), and each missed phrase counts as a false negative phrase (FN). Precision and recall were calculated using the same formulae as shown in Table 8.

Table 8: Extraction results of each concept

Concept	Ground Truth		Extracted (TP)	Matched (FP)	Misclassified (FN)	Missed	Precision	Recall
Modality	82	81	80	1	2		98.8%	97.6%
Actor	118	110	109	1	9		99.1%	92.4%
Action	147	147	145	2	2		98.6%	98.6%
Condition	46	40	40	0	6		100%	86.9%
Constraint	35	38	34	4	1		89.5%	97.1%
Exception	15	15	14	1	1		93.3%	93.3%
Overall	443	431	422	9	21		97.9%	95.3%

Overall, of the 443 extracted phrase components, 422 were correctly matched and 9 (2.1%) were misclassified. There are 21 ground-truth phrase components (4.7%) that were missed by the extraction rules due to incorrect classification or impossible classification. The impossibility of classification was mainly due to parser errors or to missing Tregex patterns (or makers) that were either too sparse or uncertain. We obtained an overall precision of 97.9% and an overall recall of 95.3%. This means that our method can extract the phrase components we need with a high precision. Therefore, it provides a more accurate semantics for compliance checking.

5.3 Evaluation of Violation Detection

The evaluation experiments in this section include two real-life cases that demonstrate the validity of the proposed matching score and the three types of violation scores in our methodology, respectively. We implemented a prototype in Python3 with a set of BPMN files, i.e., process models, as input, and a set of txt files, i.e., the text of regulatory documents, as input. Specifically, Section 5.3.1 reports experiments conducted for an Austrian energy supplier and presents the first scenario of verifying the correctness of matching score between process models and the rule records. Section 5.3.2 describes the experiment of the second scenario in the context of GDPR implementation, that is, the whole process including matching and checking of three kinds of violations.

5.3.1 Evaluation of Matching

This section demonstrates the first scenario for the process models to match the relevant rule records, thus verifying the accuracy of the matching scores.

Input. The input consists of a repository of 12 process models that were built and validated by domain experts in earlier work [33] and correspond in a one-to-many relationship to a rule base parsed from regulatory documents related to smart meters in an Austrian energy supplier.

Evaluation Strategy. We quantify our approach's accuracy using common measures for recommender systems [34]: The average precision (AP) per model, as well as the mean average precision (MAP) for the whole model repository, defined as:

$$AP@{\gamma} = \frac{1}{\Omega_m} \sum_{i \in \Omega_m} \frac{\sum_{j \in \Omega_m} h(r_{m_j} < r_{m_i}) + 1}{r_{m_i}}$$

$$MAP@{\gamma} = \frac{\sum_{m \in M} AP@{\gamma}}{|M|}$$

where Ω_m denotes the number of rule records that correctly correspond to each process model, r_{m_i} denotes the position of rule record r_i in the matching list, and $r_{m_j} < r_{m_i}$ denotes that rule record r_j is ranked before rule record r_i in the matching list.

Table 9: AP of the model at different τ and MAP of the whole system

Model	$\tau=0.0$	$\tau=0.2$	$\tau=0.4$	$\tau=0.6$	$\tau=0.8$	$\tau=0.9$
Process Model1	0.546	0.577	0.664	0.702	0.802	0.697
Process Model2	0.607	0.613	0.762	0.802	0.892	0.712
Process Model3	0.556	0.608	0.675	0.722	0.811	0.708
Process Model4	0.453	0.523	0.588	0.634	0.678	0.596
Process Model5	0.532	0.644	0.677	0.723	0.801	0.689
Process Model6	0.654	0.678	0.675	0.887	0.790	0.648
Process Model7	0.670	0.701	0.747	0.777	0.889	0.762
Process Model8	0.598	0.643	0.665	0.700	0.874	0.735
Process Model9	0.638	0.649	0.675	0.732	0.812	0.717
Process Model10	0.666	0.690	0.745	0.812	0.799	0.702
Process Model11	0.632	0.666	0.735	0.755	0.790	0.694
Process Model12	0.644	0.687	0.702	0.796	0.675	0.655
Overall(MAP)	0.600	0.640	0.693	0.754	0.801	0.693

Results. For the matching scores, the AP and MAP results are shown in Table 9. In the table, AP is used to measure each model under different values of τ , and MAP is used to measure the matching quality of the whole system under different values of τ . 9 of the 12 models had the best overall matching when $\tau = 0.8$, and 3 models had the best results when $\tau = 0.6$. It can be seen that the overall accuracy of our method is relatively high in terms of the matching results.

5.3.2 Evaluation of Checking

In this section, we thoroughly evaluate the effectiveness of our method on another real-world dataset. We use four process models as input that capture how the main privacy constraints of GDPR are enforced in processes [35].

Furthermore, as the regulatory documents, we use GDPR Articles 1 to 50, of which Articles 1 to 4 contain the introductory statement while Articles 5 to 50 contain the regulatory descriptions of the four models. In contract, Article 51 and above applies to regulators rather than business organizations and are therefore outside the scope of our checking. Based on these four process models and 50 articles, we manually constructed a collection of process models containing violations. First, the process model is supplemented to make the process model fully compliant. For the missing action violation, we delete the activity if it exists in the model. For the incorrect actor violation, if the actor is correct in the model, it is manually modified. And for the out-of-order execution violation, the positions between activities are swapped. Each checking contains only one violation. The violation data for each model is shown in Table 10.

Table 10: Details of the violation assessment dataset

Model	#Matching rules	#Missing Action	#Incorrect Actor	#Out-of-order Execution
BPMN1	21	7	5	9
BPMN2	30	9	8	13
BPMN3	38	12	10	16
BPMN4	46	23	9	16

The four process models are matched with the rule records in the rule base, and the specific results are shown in Table 11. As with the evaluation results in Section 5.3.1, the process model matches the rule records with the highest accuracy when $\tau=0.8$.

Table 11: AP of the model at different τ and MAP of the whole system

Model	$\tau=0.2$	$\tau=0.4$	$\tau=0.6$	$\tau=0.8$	$\tau=0.9$
BPMN1	0.503	0.598	0.712	0.824	0.675
BPMN2	0.599	0.637	0.785	0.855	0.713
BPMN3	0.572	0.675	0.783	0.845	0.745
BPMN4	0.523	0.600	0.755	0.835	0.722
Overall(MAP)	0.549	0.628	0.759	0.840	0.714

Due to the fact that the proposition of our method is inspired by Winter et al., which uses keyword to identify sentence types and applies all clauses as what needs to be checked without in-depth extraction of phrases, we further compared the results of this method with our method, and the results

are shown in Table 12. From the table, we can see that their method has a slightly higher recall than our method, but our method has a higher precision. The main reason for the phenomena is that their method uses all clauses for checking without filtering, which correspond to the ability to identify more violations and a higher recall. However, this practice will lead to the fact that more non-violations are detected as violations, which lowers their precision and thus significantly increases the subsequent manual checking effort reduces for the misidentification. Our approach, on the other hand, conduct effective classification on both statement level and phrase level with more advanced technique, which is able to detect violations in a more precise way. Therefore, in terms of precision and assistance with follow-up work, our strategy is superior to that of Winter et al.

Table 12: Comparison of accuracy

Methods	Precision	Recall	F-score
Winter et al. (2020)[14]	0.58	0.89	0.70
Our method	0.77	0.83	0.80

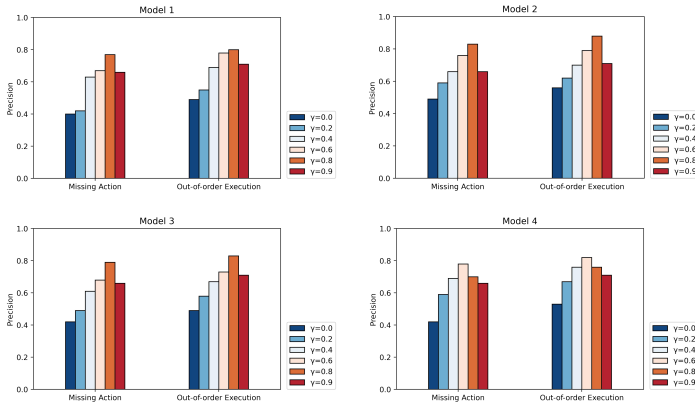


Fig. 8: Precision of each model in detecting violation for different γ

Furthermore, we examine how various thresholds affect the precision of violation detection. We analyze two categories of violations together (i.e., missing action and out-of-order execution) since the accuracy of these two categories is solely connected to the value of γ . The accuracy results of the four models at different values of γ are shown in Figure 8. From the figure, we can conclude that for most models, the highest precision of violation detection is reached when $\gamma = 0.8$. However, for model 4, the precision is highest when $\gamma = 0.6$. The reason is that model 4 has the highest complexity, which identifies more

violations when γ becomes higher, i.e., with stricter conditions, thus identifying normal operations as violations and decreasing the precision. This also explains why the precision is lower when $\gamma = 0.9$ than that when $\gamma = 0.8$.

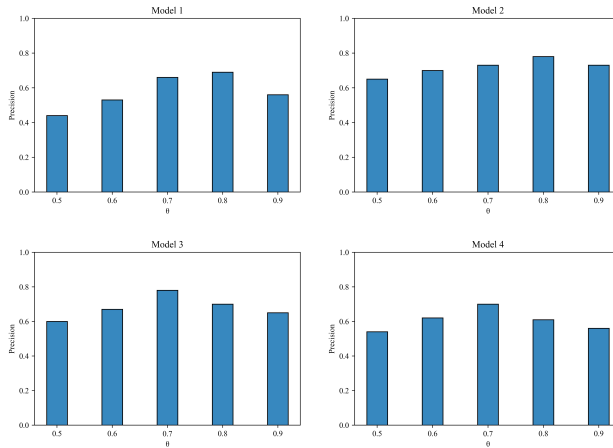


Fig. 9: Precision of each model in detecting incorrect actor violation for different ϑ

For the incorrect actor violation, its precision is influenced by both γ and ϑ , and we discuss its precision separately. According to the results of the first two violations affected by the γ , the precision is better when $\gamma=0.8$. Therefore, we explore the influence of the incorrect actor violation on the variation of ϑ when $\gamma=0.8$. The results are shown in Figure 9, with the horizontal axis indicating the value of ϑ . Two of the four models have the highest precision when $\vartheta=0.8$, and the other two have the highest precision when $\vartheta=0.7$. The reason is that the latter two models are larger with more content, the similarity of the text decreases in complex situations. Therefore, for complex process models, a lower threshold ϑ will lead to better results. Additionally, according to analysis, we found that the similarity between actors in the latter two models and those in the rules was lower, leading to a decrease in precision when the threshold was increased.

5.4 Case Study

To demonstrate the results of our entire approach more clearly, we use the example of a phone company's process for acquiring a new customer as case study in this section (Agostinelli et al., 2019). We will give a simple explanation of the business process model. First, the phone company requests new data from the customer and verifies that the data is correct. If the customer decides to change the phone number, the phone company will communicate with the

original phone company and sign a contract with the customer. After completing the above steps and receiving payment, the phone company sends the SIM card to the customer and activates it. If for any reason the process takes more than 30 days to complete, the process will be terminated. The company models the process of acquiring new customers in the design-time to obtain a BPMN model as in Figure 10, and we intend to conduct BPC checking between this model against regulatory documents from various sources, including company regulations and GDPR, a few instances of which are briefly provided Table 13.

Table 13: Rules relevant with the process from different sources

ID	Sources	Rules
R1	Company regulation	The process of phone company must be terminated if it takes more than 30 days for any reason.
R2	Company regulation	After receiving the customer's personal information, it is necessary to verify the correctness of their personal information.
R3	Company regulation	When the customer receives the sim card, the phone company must activate the sim card so that the customer can use the sim card normally.
R4	GDPR	Before retrieving any kind of personal data from the Data Subject, the Data Controller has to ask the Data Subject for consent.

The detailed BPC checking results of our method are shown in Figure 10. It can be seen that for both rules R1 and R4, we detect missing action violations. According to our results, analysts can modify the process model after the analysis. For Violation 1 with R1, a sub-process needs to be added to the process model, which terminates the process when the execution time exceeds 30 days. For Violation 4 with R4, the activity of *asking for consent* is added to the original process model. For the rule with R2, we detected the out-of-order execution violation. To address this violation, the activity of *check the correctness of personal data* should be added after the personal data has been obtained. Finally, we detect the incorrect actors violation with R3. According to this rule, the SIM card should be activated by the phone company, not the customer himself. It is worth noting that R4 is a rule from the GDPR that differs from the previous company regulation. When checking the process model, we have to manually convert the actors in the rule records because its actors are not specific but related to a class of actors. In this case, we need to convert the “Data Controller” to the “phone company” and the “Data Controller” to the “user” before checking.

6 Conclusion

In this paper, we design an automatic compliance assessment method between process models and regulatory documents, namely design-time BPC checking. To this end, we propose a general NLP approach that combines deep learning with rule-based method to extract semantic information related to

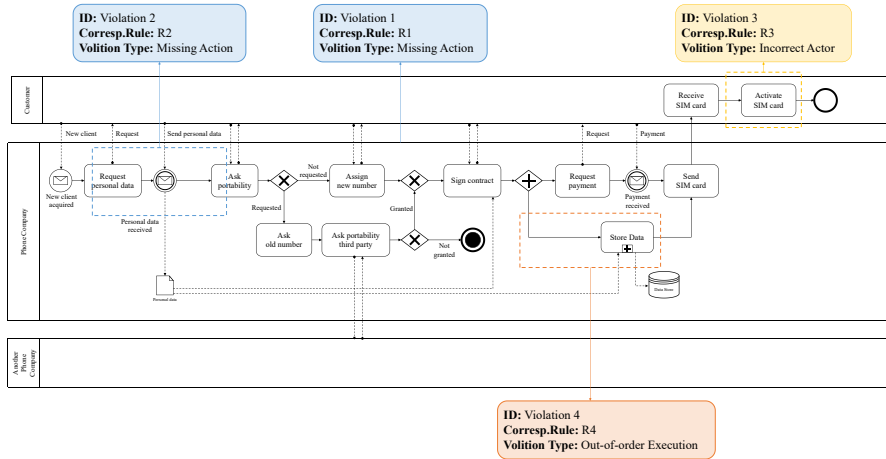


Fig. 10: Visual representation of results of process model compliance checking

process models from regulatory documents and construct a structured rule base on which to assess the compliance of process models. To improve the interpretability of our approach, we evaluate the process model from three different violation perspectives. The effectiveness of our approach has been demonstrated in two real-world datasets.

In the future, we intend to provide modification suggestions after checking violations rather than plain scores, so that process modelers can have more references for modifications and further reduce manual checking costs.

Ethical Approval

Not Applicable.

Availability of supporting data

The datasets generated during and/or analysed during the current study are available from the corresponding author on reasonable request.

Competing interests

The authors declare that they have no competing interests.

Funding

The work is supported by Natural Science Foundation of China (No.61472112), Natural Science Foundation of Zhejiang Province (No. LQ20F020017) and the Key Science and Technology Project of Zhejiang (No. 2020C01165).

Authors' contributions

Xiaoxiao Sun came up with the concept, method and wrote the main manuscript. Siqing Yang conducted the experiments, performed analysis and wrote the original draft. Chenying Zhao helped in conducting the experiments and visualizing the results. Dongjin Yu came up with the concept and supervised the whole process. All authors reviewed the manuscript.

Acknowledgments

The work is supported by Natural Science Foundation of China (No.61472112), Natural Science Foundation of Zhejiang Province (No. LQ20F020017) and the Key Science and Technology Project of Zhejiang (No. 2020C01165).

References

- [1] Van Der Aalst, W.M.: Business process management demystified: A tutorial on models, systems and standards for workflow management. In: Advanced Course on Petri Nets, pp. 1–65 (2003). Springer
- [2] Kharbili, M.e., Medeiros, A.K.A.d., Stein, S., van der Aalst, W.M.: Business process compliance checking: Current state and future challenges. *Modellierung betrieblicher Informationssysteme (MobIS 2008)* (2008)
- [3] Hashmi, M., Governatori, G., Lam, H.-P., Wynn, M.T.: Are we done with business process compliance: state of the art and challenges ahead. *Knowledge and Information Systems* **57**(1), 79–133 (2018)
- [4] Syed Abdullah, N., Sadiq, S., Indulska, M.: Emerging challenges in information systems research for regulatory compliance management. In: International Conference on Advanced Information Systems Engineering, pp. 251–265 (2010). Springer
- [5] Ly, L.T., Maggi, F.M., Montali, M., Rinderle-Ma, S., Van Der Aalst, W.M.: Compliance monitoring in business processes: Functionalities, application, and tool-support. *Information systems* **54**, 209–234 (2015)
- [6] Rinderle-Ma, S., Kabicher-Fuchs, S.: An indexing technique for compliance checking and maintenance in large process and rule repositories (2016)
- [7] Governatori, G., Sadiq, S.W.: The journey to business process compliance. In: *Handbook of Research on Business Process Modeling* (2009)
- [8] Brunello, A., Montanari, A., Reynolds, M.: Synthesis of ltl formulas from natural language texts: State of the art and research directions. In: 26th International Symposium on Temporal Representation and Reasoning (TIME 2019) (2019). Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik

- [9] Shanahan, M.: The event calculus explained. In: Artificial Intelligence Today (1999)
- [10] Elgammal, A., Turetken, O., van den Heuvel, W.-J., Papazoglou, M.: Formalizing and applying compliance patterns for business process compliance. *Software & Systems Modeling* **15**(1), 119–146 (2016)
- [11] Governatori, G., Rotolo, A.: Norm compliance in business process modeling. In: International Workshop on Rules and Rule Markup Languages for the Semantic Web, pp. 194–209 (2010). Springer
- [12] Montali, M., Maggi, F.M., Chesani, F., Mello, P., Aalst, W.M.v.d.: Monitoring business constraints with the event calculus. *ACM transactions on intelligent systems and technology (TIST)* **5**(1), 1–30 (2014)
- [13] Governatori, G., Hashmi, M., Lam, H.-P., Villata, S., Palmirani, M.: Semantic business process regulatory compliance checking using legal-ruleml. In: European Knowledge Acquisition Workshop, pp. 746–761 (2016). Springer
- [14] Winter, K., Aa, H.v.d., Rinderle-Ma, S., Weidlich, M.: Assessing the compliance of business process models with regulatory documents. In: International Conference on Conceptual Modeling, pp. 189–203 (2020). Springer
- [15] Sànchez-Ferreres, J., Burattin, A., Carmona, J., Montali, M., Padró, L.: Formal reasoning on natural language descriptions of processes. In: International Conference on Business Process Management, pp. 86–101 (2019). Springer
- [16] Delicado Alcántara, L., Sanchez-Ferreres, J., Carmona Vargas, J., Padró, L.: Nlp4bpm: natural language processing tools for business process management. In: BPM Demo and Industrial Track 2017 Proceedings, pp. 1–5 (2017)
- [17] Sinha, A., Paradkar, A.: Use cases to process specifications in business process modeling notation. In: 2010 IEEE International Conference on Web Services, pp. 473–480 (2010). IEEE
- [18] Dragoni, M., Villata, S., Rizzi, W., Governatori, G.: Combining nlp approaches for rule extraction from legal documents. In: 1st Workshop on MIning and REasoning with Legal Texts (MIREL 2016) (2016)
- [19] Selway, M., Grossmann, G., Mayer, W., Stumptner, M.: Formalising natural language specifications using a cognitive linguistic/configuration based approach. *Information Systems* **54**, 191–208 (2015)

- [20] Aa, H.v.d., Ciccio, C.D., Leopold, H., Reijers, H.A.: Extracting declarative process models from natural language. In: International Conference on Advanced Information Systems Engineering, pp. 365–382 (2019). Springer
- [21] Biagioli, C., Francesconi, E., Passerini, A., Montemagni, S., Soria, C.: Automatic semantics extraction in law documents. In: Proceedings of the 10th International Conference on Artificial Intelligence and Law, pp. 133–140 (2005)
- [22] Sleimi, A., Sannier, N., Sabetzadeh, M., Briand, L., Ceci, M., Dann, J.: An automated framework for the extraction of semantic legal metadata from legal texts. *Empirical Software Engineering* **26**(3), 1–50 (2021)
- [23] Meth, H., Brhel, M., Maedche, A.: The state of the art in automated requirements elicitation. *Information and Software Technology* **55**(10), 1695–1709 (2013)
- [24] Quishpi, L., Carmona, J., Padró, L.: Extracting annotations from textual descriptions of processes. In: International Conference on Business Process Management, pp. 184–201 (2020). Springer
- [25] Leopold, H.: Natural language in business process models. PhD thesis, Springer (2013)
- [26] McNamara, P.: Deontic logic. In: *Logic and the Modalities in the Twentieth Century* (2006)
- [27] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
- [28] Michel, M., Djurica, D., Mendling, J.: Identification of decision rules from legislative documents using machine learning and natural language processing. In: HICSS, pp. 1–10 (2022)
- [29] Li, J., Luong, M.-T., Jurafsky, D.: A hierarchical neural autoencoder for paragraphs and documents. arXiv preprint arXiv:1506.01057 (2015)
- [30] Chen, Y.: Convolutional neural network for sentence classification. Master’s thesis, University of Waterloo (2015)
- [31] Levy, R., Andrew, G.: Tregex and tsurgeon: Tools for querying and manipulating tree data structures. In: LREC, pp. 2231–2234 (2006). Citeseer
- [32] Manning, C.D., Surdeanu, M., Bauer, J., Finkel, J.R., Bethard, S.,

- McClosky, D.: The stanford corenlp natural language processing toolkit. In: Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 55–60 (2014)
- [33] Böhmer, K., Rinderle-Ma, S.: Multi-perspective anomaly detection in business process execution events. In: OTM Confederated International Conferences” On the Move to Meaningful Internet Systems”, pp. 80–98 (2016). Springer
- [34] Schröder, G., Thiele, M., Lehner, W.: Setting goals and choosing metrics for recommender system evaluations. In: UCERSTI2 Workshop at the 5th ACM Conference on Recommender Systems, Chicago, USA, vol. 23, p. 53 (2011)
- [35] Agostinelli, S., Maggi, F.M., Marrella, A., Sapio, F.: Achieving gdpr compliance of bpmn process models. In: International Conference on Advanced Information Systems Engineering, pp. 10–22 (2019). Springer